Spring: A Head Start 🔥— Building A Web Application (Part 7)

Putting all together and building a spring web application.

Omar Elgabry Follow
Sep 12, 2017 · 8 min read

Spring: A Head Start 🔥 — Building A Spring Web Application

Introduction

So we made it to the end of this tutorial. Now, It's time to put all what we've learnt so far and build a real web application implementing the CRUD operations.

We'll be building a tracking system for the *orders*. It's just a simple application to get a fair idea about:

- Setting up a web application from the scratch
- Structuring your MVC web application
- The workflow of the MVC web application
- Implementing CRUD operations
- Connecting to a database (we're using Hibernate, but, you can follow the Spring Data Access for using JDBC).
- Working with HTML form data & view pages (JSP)
- … and a lot more.

Setting Up The Environment 💪

As usual, you need to create a new "Dynamic Web Project", and add all the jar files to "WEB-INF/lib" folder, or use Maven for defining all the dependencies.

The dependencies we'll need:

- Spring Framework
- Hibernate, JDBC Driver, and C3PO (Database)
- Java Servlet + JSP + JSTL

If you're using Maven, then your "pom.xml" should have the following:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-i
<!-- .... -->
 <properties>
    <springframework.version>4.3.4.RELEASE</springframework.version>
    <hibernate.version>5.1.7.Final</hibernate.version>
```

```
    <mysql.connector.version>5.1.40</mysql.connector.version>
 </properties>
<dependencies>
      <!-- Apache Common Logging API -->
      <dependency>
         <groupId>commons-logging</groupId>
         <artifactId>commons-logging</artifactId>
         <version>1.2</version>
      </dependency>
        <!-- Spring -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-webmvc</artifactId>
            <version>${springframework.version}</version>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-tx</artifactId>
            <version>${springframework.version}</version>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-orm</artifactId>
            <version>${springframework.version}</version>
        </dependency>
        <!-- Hibernate -->
        <dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>${hibernate.version}</version>
        </dependency>

        <!-- MySQL -->
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>${mysql.connector.version}</version>
        </dependency>

        <!-- C3PO -->
        <dependency>
            <groupId>com.mchange</groupId>
            <artifactId>c3p0</artifactId>
            <version>0.9.2.1</version>
```

```
        </dependency>
        <!-- Servlet+JSP+JSTL -->
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>3.1.0</version>
        </dependency>
        <dependency>
            <groupId>javax.servlet.jsp</groupId>
            <artifactId>javax.servlet.jsp-api</artifactId>
            <version>2.3.1</version>
        </dependency>
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>jstl</artifactId>
            <version>1.2</version>
        </dependency>
    </dependencies>
  <build> <!-- ... --> </build>
  </project>
```

```
CREATE TABLE `orders` (
 `id` int(11) NOT NULL AUTO_INCREMENT,
 `customer` varchar(60) NOT NULL,
 `price` int(11) NOT NULL,
 `date_of_order` date NOT NULL,
 PRIMARY KEY (`id`)
)
```

orders.sql hosted with ♡ by GitHub view raw

Configurations

Dispatcher Servlet(s)

Create an XML file; “web.xml”, for defining and configuring Spring MVC dispatcher servlet(s).

> *We can add a welcome page. So, whenever we enter the root URL, the server will look for the welcome page file (if defined) in the root directory of the web application:*

> *"WebContent/index.jsp".*

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns="http://xmlns.jcp.org/xml/ns/javaee"
 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-a
 id="WebApp_ID" version="3.1">

<display-name>OrdersTracker</display-name>

<!-- Spring MVC Configs -->

<!-- Welcome Page -->
  <welcome-file-list>
     <welcome-file>index.jsp</welcome-file>
   </welcome-file-list>

 <!-- Configure Spring MVC Dispatcher Servlet -->
 <servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
   <param-name>contextConfigLocation</param-name>
   <param-value>/WEB-INF/spring-mvc-demo-servlet.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
 </servlet>

<!-- Set up URL mapping for Spring MVC Dispatcher Servlet -->
 <servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>/</url-pattern>
 </servlet-mapping>
 </web-app>
```

The "dispatcher" Dispatcher Servlet

The configurations for our dispatcher is in"WEB-INF/spring-mvc-demo-servlet.xml" file.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
 xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
 xmlns:mvc="http://www.springframework.org/schema/mvc"
 xsi:schemaLocation="
  http://www.springframework.org/schema/beans
  http://www.springframework.org/schema/beans/spring-beans.xsd
  http://www.springframework.org/schema/context
  http://www.springframework.org/schema/context/spring-context.xsd
  http://www.springframework.org/schema/mvc
  http://www.springframework.org/schema/mvc/spring-mvc.xsd
  http://www.springframework.org/schema/tx
  http://www.springframework.org/schema/tx/spring-tx.xsd">

<!-- Add support for component scanning -->
 <context:component-scan base-package="com.demo.mvc" />

<!-- Add support for conversion, formatting and validation support -->
 <mvc:annotation-driven/>

<!-- Define Spring MVC view resolver -->
 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/view/" />
  <property name="suffix" value=".jsp" />
 </bean>

 <!-- A handler for serving static resources -->
 <mvc:resources mapping="/assets/**" location="/assets/" />

 </beans>
```

Database Configurations

In this application, we are going to use Hibernate.

Instead of using “hibernate.cfg.xml” file, the configurations for connecting to database can be defined as a *DataSource* bean in the XML configuration file.

```
<!-- Step 1: Define Database DataSource -->
 <bean id="myDataSource" class="com.mchange.v2.c3p0.ComboPooledDataSource" destroy-method="close
  <property name="driverClass" value="com.mysql.jdbc.Driver"/>
  <property name="jdbcUrl" value="jdbc:mysql://localhost/demoDB"/>
  <property name="user" value="root"/>
  <property name="password" value=""/>
```

```
 <!-- These are connection pool properties for C3P0 (Connection Pool) -->
   <property name="initialPoolSize" value="5"/>
   <property name="minPoolSize" value="5" />
   <property name="maxPoolSize" value="20" />
   <property name="maxIdleTime" value="30000" />
  </bean>
```

web.xml hosted with ♡ by GitHub view raw

> *Hibernate make use of connection pool for connecting to the database. We're using a third-party software for connection pooling.*

Remember when we had to create a session object using SessionFactory? — Now, we can define *SessionFactory* as a bean in the XML configuration file, which has *DataSource* as a dependency.

```
 <!-- Step 2: Setup Hibernate session factory -->
  <bean id="sessionFactory" class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
   <property name="dataSource" ref="myDataSource" />
   <property name="packagesToScan" value="com.demo.mvc.entities" />
   <property name="hibernateProperties">
    <props>
      <prop key="hibernate.dialect">org.hibernate.dialect.MySQLDialect</prop>
      <prop key="hibernate.show_sql">true</prop>
    </props>
   </property>
  </bean>
```

web.xml hosted with ♡ by GitHub view raw

Remember when we had to write code for starting and close transaction? — This can be eliminated by defining a bean for transaction manager (which has *SessionFactory* as a dependency), and enabling `@Transactional` annotation, unless you want to do the transaction code (begin, end, etc) manually.

```
 <!-- Step 3: Setup Hibernate transaction manager -->
  <bean id="myTransactionManager" class="org.springframework.orm.hibernate5.HibernateTransactionMa
     <property name="sessionFactory" ref="sessionFactory"/>
  </bean>
```

```
<!-- Step 4: Enable configuration of transactional behavior based on annotations -->
<tx:annotation-driven transaction-manager="myTransactionManager" />
```

web.xml hosted with ♡ by GitHub view raw

This annotation auto-magically begin and end the transaction in your Hibernate code (as we'll see later).

The Structure

There are many ways to structure your MVC application, and add layers as a way to segregate operations and tasks.

One of the most common ways is to use *DAO Layer* to perform database operations and map the result into objects (entities / DTOs).

> *The DTOs (Data Transfer Object) are simple objects that carries data between processes. DTOs are often used in conjunction with data access objects to retrieve data from a database.*

The *Service Layer* is an intermediate layer between the controller and the DAO layer. It calls the DAO and perform for custom business logic. For example, It can integrate data from different resources (i.e. repositories, DAO, etc).

Packaging

So, first, let's organize classes into groups of related types into packages as a way to avoid naming conflicts, and to control access.

> *"types" refers to classes, interfaces, enumerations, and annotation types.*

Create a package "*com.demo.mvc*" in "src" folder, and inside this package create:

- **Controllers**: "*com.demo.mvc.controllers*".
- **DAO**: "*com.demo.mvc.dao*".
- **Services**: "*com.demo.mvc.services*".
- **Entities**: "*com.demo.mvc.entities*".

> 💡 *Controller classes will go under com.demo.mvc.controllers package, and so on.*

The Workflow

The workflow is as the following:

- The user sends a request from the browser.
- The dispatcher recieves the request and forward it to the corresponding controller method.
- The controller talk to the service layer to complete the required work (i.e. ask for some data).
- The service layer talk to DAOs for database operations and perform business operations.
- The DAO layer performs database operations using map the data returned with Entities / DTOs of the application.
- Now, In the controller, we can build the model object and return a view page to be displayed, redirect to another page, or return JSON data, etc.

Entities — Order

The order is an entity which represents the *orders* the database table. The data returned from the database will be mapped to order objects.

```
package com.demo.mvc.entities;

import java.util.Date;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="orders")
public class Order {

 @Id
```

```
  @GeneratedValue(strategy=GenerationType.IDENTITY)
  @Column(name="id")
  private int id;

  private String customer;

  private Integer price;

  @Column(name="date_of_order")
  private Date dateOfOrder;

  public Order(){
  }
  /* ... */
 }
```

> ⚠ *Don't forget to define getters and setters for class object fields.*
>
> 💡 `GenerationType.IDENTITY` *means this field will be auto-incremented and will get assigned an id automatically whenever we save a new order.*
>
> 💡 *The classes with* `@Entity` *annotation in* `com.demo.mvc.entities` *package will be scanned for mapping, and creating the session object.*

DAO — OrderDAO & OrderDAOImpl

The *OrderDAO* interface provides a set of methods (list, add, update, delete) operations to be implemented by *OrderDAOImpl* class.

```
package com.demo.mvc.dao;

import java.util.List;
import com.demo.mvc.entities.Order;

public interface OrderDAO {

 public List<Order> getOrders();
 public void saveOrder(Order newOrder);
 public Order getOrder(int orderId);
 public void deleteOrder(int orderId);
}
```

OrderDAO.java hosted with by GitHub view raw

The code implementation for *OrderDAOImpl* will be explained later in the list, add, update, and delete operations. But, for now, lets see it's basic structure.

```
package com.demo.mvc.dao;

import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
import com.demo.mvc.dao.OrderDAO;
import com.demo.mvc.entities.Order;

@Repository
public class OrderDAOImpl implements OrderDAO {

  @Autowired
  private SessionFactory factory;

  @Override
  public List<Order> getOrders(){
   // get the current hibernate session
   Session session = factory.getCurrentSession();

   // execute query and get result list

   // return result
  }

  @Override
  public void saveOrder(Order newOrder) { /* ... */ }

  @Override
  public Order getOrder(int orderId) { /* ... */ }

  @Override
  public void deleteOrder(int orderId) { /* ... */ }
}
```

The `@Repository` annotation inherits from `@Component` annotation (same as `@Controller`). It used to identify the implementing class of DAO. This allows for auto scanning for these classes (thanks to component scanning defined in configuration), and any checked exception that will be thrown are translated into unchecked exceptions.

The DAO implementing classes needs a Hibernate *SessionFactory* object, which in turn needs a *DataSource* as a dependency (defined in configuration).

Service — OrderService & OrderServiceImpl

The *OrderService* interface has a set a methods that almost similar to methods in *OrderDAO* interface, and will be implemented by *OrderServiceImpl* class.

```
package com.demo.mvc.services;

import java.util.List;
import com.demo.mvc.entities.Order;

public interface OrderService {

 public List<Order> getOrders();
 public void saveOrder(Order newOrder);
 public Order getOrder(int orderId);
 public void deleteOrder(int orderId);
}
```

OrderService.java hosted with ♡ by GitHub view raw

The implementation for *OrderServiceImpl* in our example is fairly simple. The service layer needs to talk to DAO layer and ask for the requested data.

```
package com.demo.mvc.services;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.demo.mvc.dao.OrderDAO;
import com.demo.mvc.entities.Order;
import com.demo.mvc.services.OrderService;

```

```
@Service
public class OrderServiceImpl implements OrderService {

  @Autowired
  private OrderDAO orderDAO;

  @Override
  @Transactional
  public List<Order> getOrders() {
   return orderDAO.getOrders();
  }

  @Override
  @Transactional
  public void saveOrder(Order newOrder) {
    orderDAO.saveOrder(newOrder);
  }

  @Override
  @Transactional
  public Order getOrder(int orderId) {
    return orderDAO.getOrder(orderId);
  }

  @Override
  @Transactional
  public void deleteOrder(int orderId) {
    orderDAO.deleteOrder(orderId);
  }
}
```

The `@Service` annotation inherits from `@Component` annotation (same as `@Controller` and `@Repository`). It used to identify the Service classes. It will be automatically scanned and registered as a bean thanks to component scanning.

The `@Transactional` annotation (configured and enabled in the XML configuration file) takes care of all transaction code needs to interact with the database in Hibernate.

> *If all class methods will run inside a transaction, then you can annotate the whole class with `@Transactional` annotation.*

📢 One may ask a question, *"Why we use this annotation in the Service layer instead of DAO layer? "*

Because we want the DAO classes to run within the transaction defined by the service layer. So, a service will start transaction, make calls to DAO class and other resources (if needed), and then end the transaction.

Controller — OrderController

```
package com.demo.mvc.controllers;

import java.util.List;
import javax.servlet.http.HttpServletRequest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import com.demo.mvc.entities.Order;
import com.demo.mvc.services.OrderService;

@Controller
@RequestMapping("/order")
public class OrderController {

  @Autowired
  private OrderService orderService;

  @GetMapping("/list")
  public String list(Model model){ /* ... */ }

  @GetMapping("/newOrder")
  public String newOrder(Model model){ /* ... */ }

  @PostMapping("/saveOrder")
  public String saveOrder(@ModelAttribute("order") Order order){ /* ... */ }

  @GetMapping("/viewOrder")
  public String viewOrder(@RequestParam("orderId") int orderId, Model model){ /* ... */ }

```

```
  @GetMapping("/deleteOrder")
  public String deleteOrder(@RequestParam("orderId") int orderId, Model model){ /* ... */ }
}
```

> *The `@GetMapping` annotation is a shortcut for `@RequestMapping(method = RequestMethod.GET)`. The same idea applies for `@PostMapping` annotation.*

Implementing The Operations — List

The first operation is to list all the orders. In the DAO layer, we will execute the query and return the result.

```
@Override
public List<Order> getOrders() {

  // get the current hibernate session
  Session session = factory.getCurrentSession();

  // create a query to select all orders
  Query theQuery = session.createQuery("from Order");

  // execute query and get result list
  List<Order> orders = theQuery.list();

  // return the result
  return orders;
}
```

OrderDAOImpl.java hosted with ♡ by GitHub view raw

In the Service layer, just make a call to `getOrders()` method using the autowired `orderDAO` (dependency).

```
@Override
@Transactional
public List<Order> getOrders() {
  return orderDAO.getOrders();
}
```

OrderServiceImpl.java hosted with ♡ by GitHub view raw

In the controller, also make a call `getOrders()` method of the Service layer. Then, add the orders to the model's attribute to be displayed in view page.

```
@GetMapping("/list")
public String list(Model model){

  // get orders from dao
  List<Order> orders = orderService.getOrders();

  // add the orders to the model
  model.addAttribute("orders", orders);

  return "list-orders";
}
```

OrderController.java hosted with ♡ by GitHub view raw

> *We've defined in the configurations that all views page will be located in "/WEB-INF/view" folder. So, create "list-orders.jsp" page inside view folder.*

The view page will simply loop over collection of orders using *forEach* tag.

> *One of the JSTL core tags (discussed before) is the iteration tag `<c:forEach>` with which we can loop over collection of items in our JSP pages.*

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose
<html>
<head>
 <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
 <title>Orders Tracker</title>
</head>
<body>
   <table>
     <tr>
       <th>Customer</th>
       <th>Price</th>
       <th>Date of Order</th>
     </tr>

```

```
      <c:forEach var="order" items="${orders}">
       <tr>
          <td> ${order.customer} </td>
          <td> ${order.price} </td>
          <td> ${order.dateOfOrder} </td>
       </tr>
     </c:forEach>
    </table>
  </body>
  </html>
```

Implementing The Operations — Add

The next operation is to add a new order. This can be done through a form submission.

So, basically we need to do two operations. In the first time, an empty form will be displayed, and then, we process the submitted data.

In the DAO layer, we will add a new order record to the database.

```
@Override
public void saveOrder(Order newOrder) {

  // get the current hibernate session
  Session session = factory.getCurrentSession();

  // create the order object to the database table
  session.save(newOrder);
}
```

OrderDAOImpl.java hosted with ♡ by GitHub view raw

In the service layer, just call the `saveOrder()` method same as before.

```
@Override
@Transactional
public void saveOrder(Order newOrder) {
  orderDAO.saveOrder(newOrder);
}
```

OrderServiceImpl.java hosted with ♡ by GitHub view raw

In the controller, we need first to create a model object, and pass a new empty object of a order (using no argument constructor).

```
@GetMapping("/newOrder")
public String newOrder(Model model){
  model.addAttribute("order", new Order());
  return "order-form";
}
```

OrderController.java hosted with ♡ by GitHub view raw

This object will be used to bind it with data when the form is submitted. So, we can fetch this object once form is submitted and save it to the database.

```
@PostMapping("/saveOrder")
public String saveOrder(@ModelAttribute("order") Order order){

  // create the order using the service
  orderService.saveOrder(order);

  return "redirect:/order/list";
 }
```

OrderController.java hosted with ♡ by GitHub view raw

For the view page, we need to create a page with HTML form for creating a new order. So, create “order-form.jsp” page in *view* folder.

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose
<html>
<head>
 <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
 <title>Orders Tracker</title>
</head>
<body>

 <h3>Add Order</h3>

  <form:form action="saveOrder" modelAttribute="order" method="POST">
```

```
      <label>Customer:</label>
      <form:input path="customer"/> <br/>

      <label>Price:</label>
      <form:input path="price"/> <br/>

      <label>Date of Order:</label>
      <form:input path="dateOfOrder"/> <br/>

      <input type="submit" value="Save" class="save" />
    </form:form>
  </body>
  </html>
```

> ⚠ Since the data submitted is string. So, for dates, we need to map between the submitted string and actual date value.

The solution is, either use InitBinder, or @DateTimeFormat annotation. In case of using the annotation:

— [1] Add `@DateTimeFormat(pattern = "MM-dd-yyyy")` to *dateOfOrder* in Order class.

```
@Column(name="date_of_order")
@DateTimeFormat(pattern = "MM-dd-yyyy")
private Date dateOfOrder;
```

— [2] Change the form input field of date in “order-form.jsp” to:

```
<input type="text" path="dateOfOrder" name="dateOfOrder" value = "
<fmt:formatDate value="${order.dateOfOrder}" pattern="MM-dd-yyyy"
/>"/>
```

— [3] Add reference to The JSTL formatting tags

It’s used to format and display text, the date, the time, and numbers.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt"  %>
```

> ⚠ *You have to use the same format (MM-dd-yyyy), Otherwise use InitBinder.*

Implementing The Operations — Update

The update operation is not so different from the Add operation. It also relies on a form submission.

So, also, there are two operations. The first, a form will be populated with order data to be updated (given order id), then, process the submitted data.

In the DAO layer, first, we need to fetch an order object given it's id.

```java
@Override
public Order getOrder(int orderId) {

  // get the current hibernate session
  Session session = factory.getCurrentSession();

  // get the order using the passed id (primary key)
  return session.get(Order.class, orderId);
}
```

OrderDAOImpl.java hosted with ♡ by GitHub view raw

Then, we update an existing order object. For this, we just need to add a small modification to `saveOrder()` method to make it works in both cases; creating and updating an existing order .

```java
@Override
public void saveOrder(Order newOrder) {

  // get the current hibernate session
  Session session = factory.getCurrentSession();

  // create/update the order object to the database table
  session.saveOrUpdate(newOrder);
}
```

OrderDAOImpl.java hosted with ♡ by GitHub view raw

> 💡 *The `saveOrUpdate()` method will add a new record if order object has no id (primary key), otherwise, it will update the given order in the table.*

Similarly, In the Service layer, we have two operations; get a specific order given it's id, and update an existing order.

For update, use the same method `saveOrder()` for creating and updating an existing order.

```
@Override
@Transactional
public Order getOrder(int orderId) {
  return orderDAO.getOrder(orderId);
}

@Override
@Transactional
public void saveOrder(Order newOrder) { /* ... */ }
```

OrderServiceImpl.java hosted with ♡ by GitHub view raw

In the controller, we need first to display the HTML form populated with order data to be updated (given order id). The order id can be sent in the URL parameter (i.e. */order/viewOrder?orderId=125*).

We'll use the same HTML form we used before for creating a new order, thus, the submitted data will hit the same method `saveOrder()` we used for creating a new order. This method will behave the same in both ways.

```
@GetMapping("/viewOrder")
public String viewOrder(@RequestParam("orderId") int orderId, Model model){

  // get the order using the service
  Order order = orderService.getOrder(orderId);

  // set order as a model attribute to pre-populate the form
  model.addAttribute("order", order);

  return "order-form";
}
```

```
@PostMapping("/saveOrder")
public String saveOrder(@ModelAttribute("order") Order order){ /* ... */ }
```

OrderController.java hosted with ♡ by GitHub view raw

In the view pages, we need to first, add an update link to each order in the “list-orders” page.

```
<table>
    <tr>
      <th>Customer</th>
      <th>Price</th>
      <th>Date of Order</th>
      <th>Actions</th>
    </tr>

    <c:forEach var="order" items="${orders}">
     <tr>
        <td> ${order.customer} </td>
        <td> ${order.price} </td>
        <td> ${order.dateOfOrder} </td>
        <td>
          <a href="/order/viewOrder?orderId=${order.id}">Update</a>
        </td>
     </tr>
   </c:forEach>
</table>
```

list-orders.jsp hosted with ♡ by GitHub view raw

And, In the “order-form.jsp” view page, add a hidden input form field in the HTML form for the order id. So when the form is submitted, we know which order to be updated.

```
<form:hidden path="id" />
```

Implementing The Operations — Delete

The delete operation should seem easy now. All what we need to do is to delete an existing order given it’s id in the URL parameter.

In the DAO layer, delete an order given it's id.

```
@Override
public void deleteOrder(int orderId) {

  // get the current hibernate session
  Session session = factory.getCurrentSession();

  // delete the order using the passed id (primary key)
  Query theQuery = session.createQuery("delete from Order where id=:Id");
  theQuery.setParameter("Id", orderId);
  theQuery.executeUpdate();
}
```

OrderDAOImpl.java hosted with ♡ by GitHub view raw

In the Service layer, as usual, make a call to `deleteOrder()` method.

```
@Override
@Transactional
public void deleteOrder(int orderId) {
  orderDAO.deleteOrder(orderId);
}
```

OrderServiceImpl.java hosted with ♡ by GitHub view raw

In the controller, call `deleteOrder()` of that Service layer to delete the given order, and redirect to list all orders page.

```
@GetMapping("/deleteOrder")
public String deleteOrder(@RequestParam("orderId") int orderId, Model model){

  // delete the order using the service
  orderService.deleteOrder(orderId);

  return "redirect:/order/list";
 }
```

OrderController.java hosted with ♡ by GitHub view raw

In the view page “list-orders.jsp”, we need to add a delete link to each displayed order, same as we did with update.

```
<table>
    <tr>
      <th>Customer</th>
      <th>Price</th>
      <th>Date of Order</th>
      <th>Actions</th>
    </tr>

    <c:forEach var="order" items="${orders}">
     <tr>
        <td> ${order.customer} </td>
        <td> ${order.price} </td>
        <td> ${order.dateOfOrder} </td>
        <td>
          <a href="/order/viewOrder?orderId=${order.id}">Update</a>
          | <a href="/order/deleteOrder?orderId=${order.id}" onclick="if(!confirm('Are you sure
        </td>
     </tr>
   </c:forEach>
</table>
```

list-orders.jsp hosted with ♡ by GitHub

Thank you for reading! If you enjoyed it, please clap 👏 for it.

Java Spring Framework Programming Software Development Other

About Help Legal

Get the Medium app

Download on the App Store
GET IT ON Google Play